

NANYANG TECHNOLOGICAL UNIVERSITY

Example Class

Prof Douglas Maskell

asdouglas@ntu.edu.sg

N4-02c-98

School of Computer Engineering

NTU

Course Site: https://ntulearn.ntu.edu.sg/

Schedule

Week	Topic	Tutorial	Lecture 1 (Monday 8:30)	Lecture 2 (Wednesday 8:30)
1	Mod 1 (Intro, Review Verilog, Design Flow, IP)	No Tut	Introduction	Not used
2	Mod 2 (Verification & Verilog Testbenches)	Tut 1	Not Used	Not used
3	Mod 3 (FPGA Architecture and Synthesis)	Tut 2	Not used	Not Used
4	Mod 4 (Arithmetic on FPGAs)	Tut 3	Not used	**EC1** (Mod 1 & 2)
5	Mod 5 (Timing and Pipelining)	Tut 4	Not Used	**EC2** (Mod 3 & 4)
6	Mod 5 (Timing and Pipelining) continued	Tut 5	Not used	Not used
7	Mod 7 (**eLearning** Buses & interfacing)	No Tut	Not used	**QUIZ**
Recess				
8	Mod 8 (uP Case Study)	No Tut	Not Used	**EC3** (Mod 5 & 7)
9	Mod 8 (uP Case Study)	Tut 7	Not used	Not used
10	Mod 9 (Asynchronous Design)	Tut 8	Not used	**EC4** (Mod 8)
11	Mod 9 (Asynchronous Design)	Tut 9	Not Used	Not used
12	--	Tut 10	Not used	**EC5** (Mod 9)
13	--	Not used	Not used	Not used

The yellow shaded regions are for Example Classes (which are in addition to the LAMS-based lectures).
The green shaded region is for a class quiz (details will be provided later).
There is no Tutorial 6 as this has been integrated into the eLearning module.
The current Lab schedule can be accessed from: https://freeaccess.scse.ntu.edu.sg/scselabs

This may change slightly throughout the semester
Individual Tutors are likely to change the tutorial timing.



NANYANG TECHNOLOGICAL UNIVERSITY

Example Class

Module 8 (EC4)

uP case study

Summary (1)

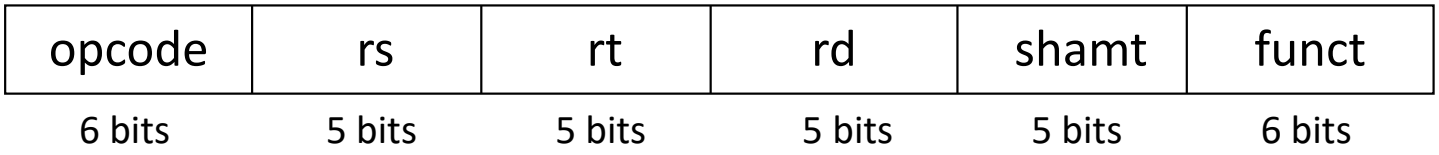
- Controller implementations
  - Simple decoders through FSMs to Microcode Controllers
- The MIPS Processor
  - 3 different instruction types: R-type, I-type & J-type
  - Looked at what components we needed
  - Arrived at the basic un-pipelined processor
- The uP16 processor
  - We examined a 16-bit processor, and what changes were necessary because of the 16-bit restriction.
  - We next looked at the Verilog code for the major components
  - Then, we saw that different instructions used different blocks in the datapath and took different times to execute.

Summary (2)

- Pipelining
  - How to add registers into the processor datapath
  - Need to forward signals to the various blocks via the registers
  - Needed to modify the design to handle clocked memories and the clocked register file
  - Used the 2 parity bits to give an 18-bit instruction memory
  - Looked at the controller implementation
- Pipeline hazards
  - Structural hazards
    - Examined why the register file did not represent a structural hazard
  - Control hazards
    - Branch instructions
  - Data hazards
    - Examined how to forward data from the pipeline to the EX stage
    - Examined when forwarding will not work (Branch, Memory access)
    - Looked at how to add data forwarding into the datapath

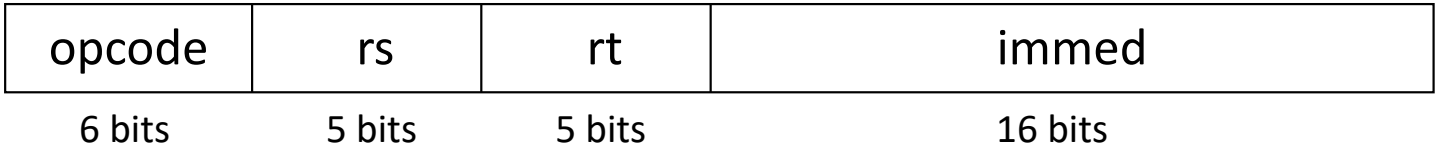
MIPS Instruction Fields

- R Type

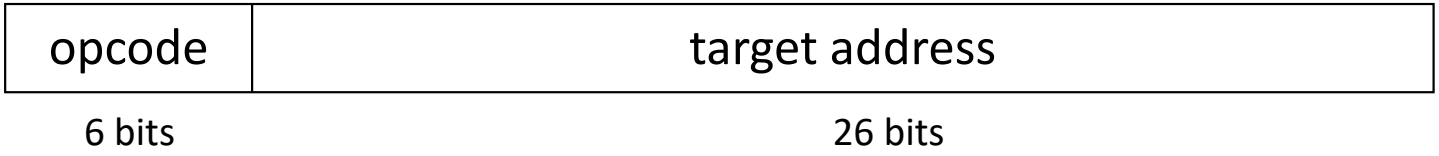


Arith. & logical instructs. have the same ***op*** (0x00) but different ***funct***

- I Type



- J Type



  - *opcode:* The opcode indicates the basic operation of the instruction
  - *rs:* The first register source operand
  - *rt:* The second register source operand
  - *rd:* The register destination operand. The result of the operation.
  - *shamt:* Shift amount (0 if not shift instruction)
  - *funct:* Function. Specifies the variant of the operation in the op field.
  - *immed:* The immediate value (arithmetic) or address offset (memory)
  - *target:* The target address for absolute jump

Word size restrictions

- MIPS **add** *r1, r2, r3* // RF[r1] <= RF[r2] + RF[r3];

R-type

op	rs	rt	rd	shamt	funct
6 bits	5 bits	5 bits	5 bits	5 bits	6 bits

I-type

op	rs	rt	immed
6 bits	5 bits	5 bits	16 bits

J-type

opcode	target address
6 bits	26 bits

- uP16 **add** *r1, r2* // RF[r1] <= RF[r1] + RF[r2];

R-type

op	rd	rs	sh/funct/immed
4 bits	3 bits	3 bits	~~6 bits~~ **8 bits**

J-type

op	target
4 bits	~~12 bits~~ **14 bits**

18-bit Instruction

uP16 Instruction set

Name	op	Funct/ Immed	Instruction	Effect(s)
nop	0x00	0x00	**nop**	Do nothing (instruction word = 0)
add	0x00	0x01	**add** *rd, rs*	RF[rd] = RF[rd] + RF[rs];
sub	0x00	0x02	**sub** *rd, rs*	RF[rd] = RF[rd] - RF[rs];
and	0x00	0x03	**and** *rd, rs*	RF[rd] = RF[rd] **and** RF[rs];
or	0x00	0x04	**or** *rd, rs*	RF[rd] = RF[rd] **or** RF[rs];
nand	0x00	0x05	**nand** *rd, rs*	RF[rd] = RF[rd] **nand** RF[rs];
nor	0x00	0x06	**nor** *rd, rs*	RF[rd] = RF[rd] **nor** RF[rs];
xor	0x00	0x07	**xor** *rd, rs*	RF[rd] = RF[rd] **xor** RF[rs];
not	0x00	0x08	**not** *rd, rs*	RF[rd] = **not** RF[rs];
mov	0x00	0x0A	**mov** *rd, rs*	RF[rd] = RF[rs];
lw	0x01	--	**lw** *rd, rs, immed*	RF[rd] <- memory(RF[rs]+immed);
sw	0x02	--	**sw** *rd, rs, immed*	RF[rd] -> memory(RF[rs]+immed);
lli	0x03	immed	**lli** *rd, rs, immed*	RF[rd] = sign_ext(immed); (*rs* is unused)
lui	0x04	immed	**lui** *rd, rs, immed*	RF[rd] = {immed, RF[rs][7:0]}
addi	0x05	immed	**addi** *rd, rs, immed*	RF[rd] = RF[rs] + sign_ext(immed);
	0x6-8			**Reserved**
beq	0x08	immed	**beq** *rd, rs, immed*	if (RF[rd] = = RF[rs]) pc += sign_ext(immed) + 1;
bne	0x09	immed	**bne** *rd, rs, immed*	if (RF[rd] != RF[rs]) pc += sign_ext(immed) + 1;
blt	0x0A	immed	**bltz** *rd, rs, immed*	if (RF[rd] < RF[rs]) pc += sign_ext(immed) + 1;
bgt	0x0B	immed	**bgtz** *rd, rs, immed*	if (RF[rd] > RF[rs]) pc += sign_ext(immed) + 1;
	0xC-F			**Reserved**

The non-pipelined processor

- It takes a long time to complete a single instruction!
- All instructions take as much time as the slowest!

Arithmetic & Logical

PC	Inst Memory	Reg File	ALU	Write

Load

PC	Inst Memory	Reg File	ALU	Data Mem	Write

Critical Path

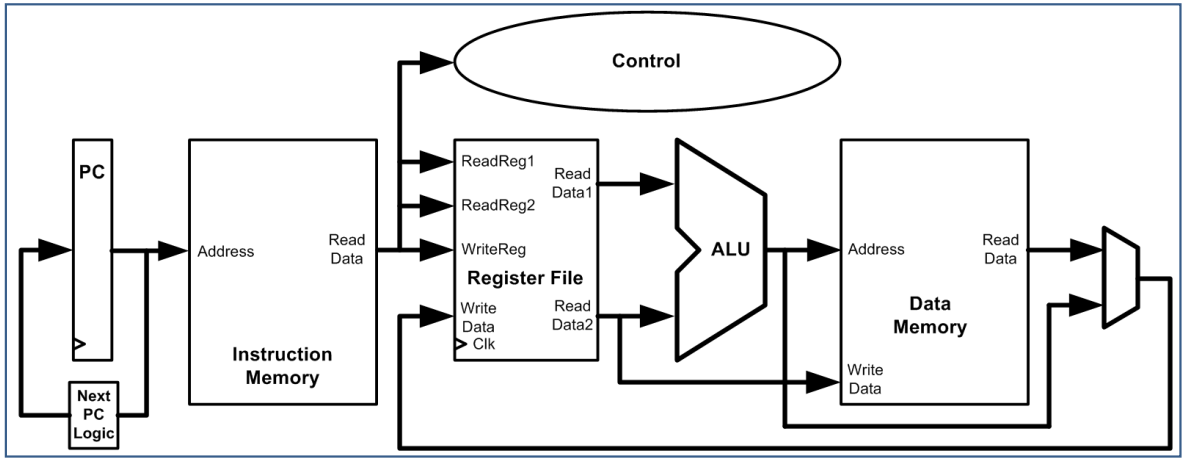
Store

PC	Inst Memory	Reg File	ALU	Data Mem

Branch

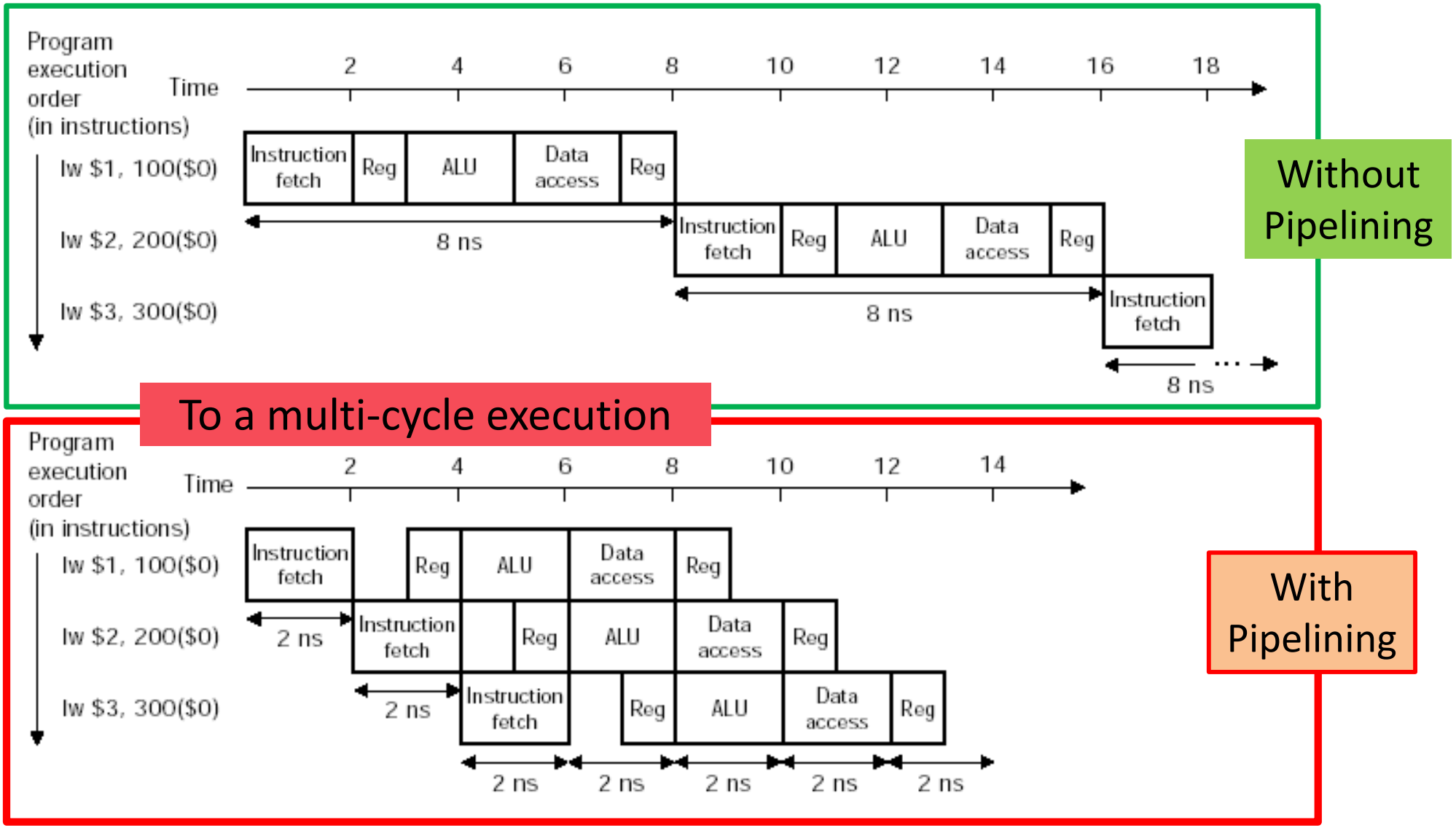
PC	Inst Memory	Reg File	Mux

Note in uP16 we make the branch decision in the ID stage



Hence the ***lw*** instruction determines how fast we can clock the processor.

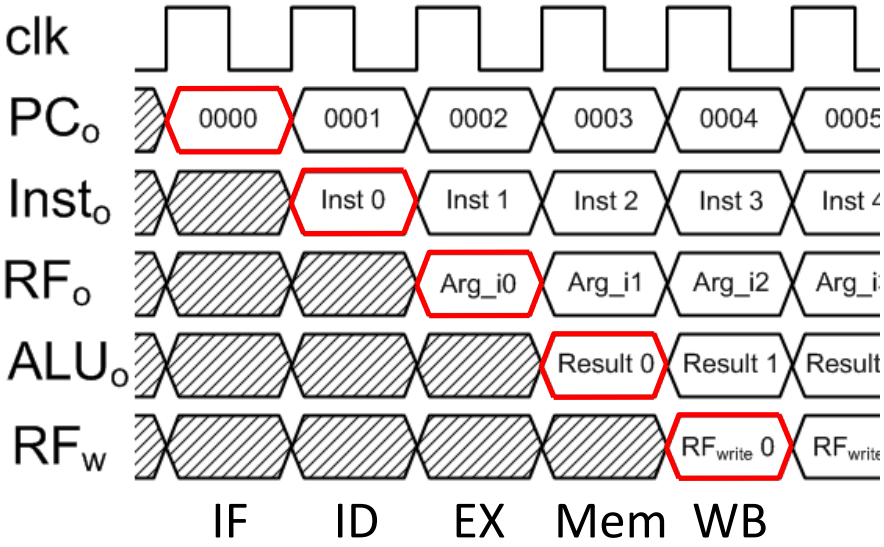
The pipelined processor

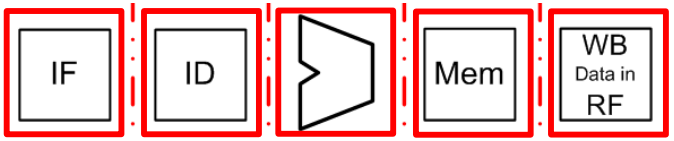


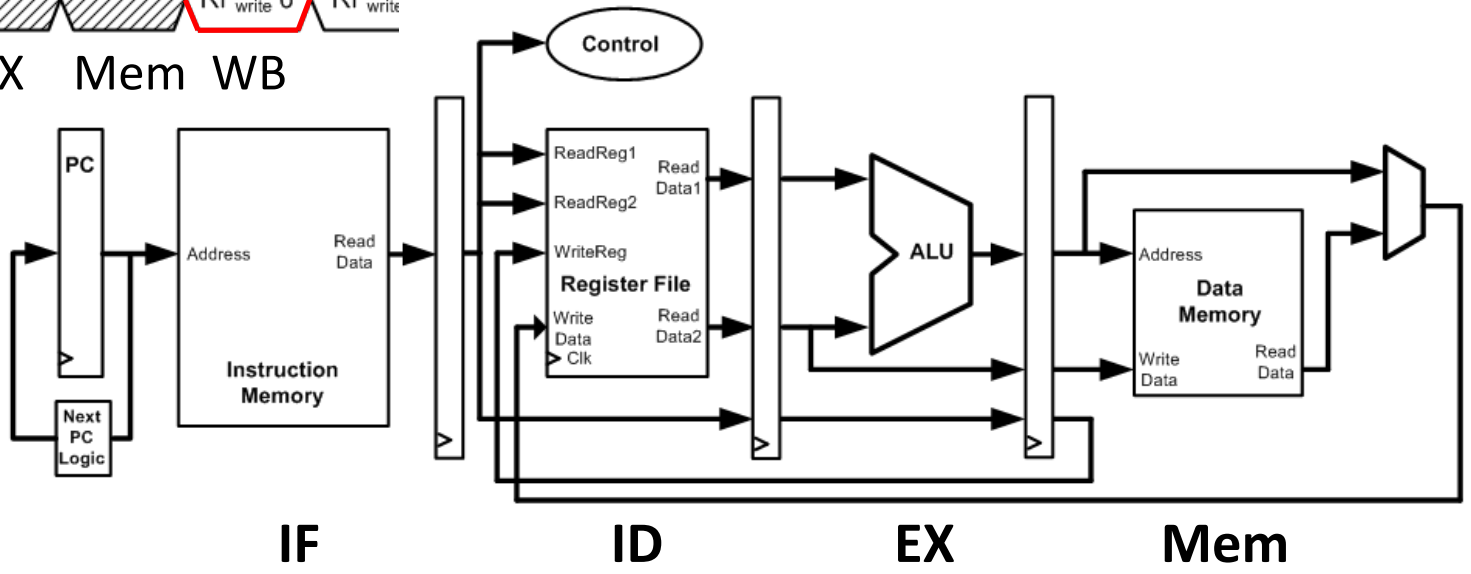
The pipelined processor

18 bits

Field	4 bits	3 bits	3 bits	8 bits
R-format	OpCode	Rd	Rs	Function/Immed8/Displacement8

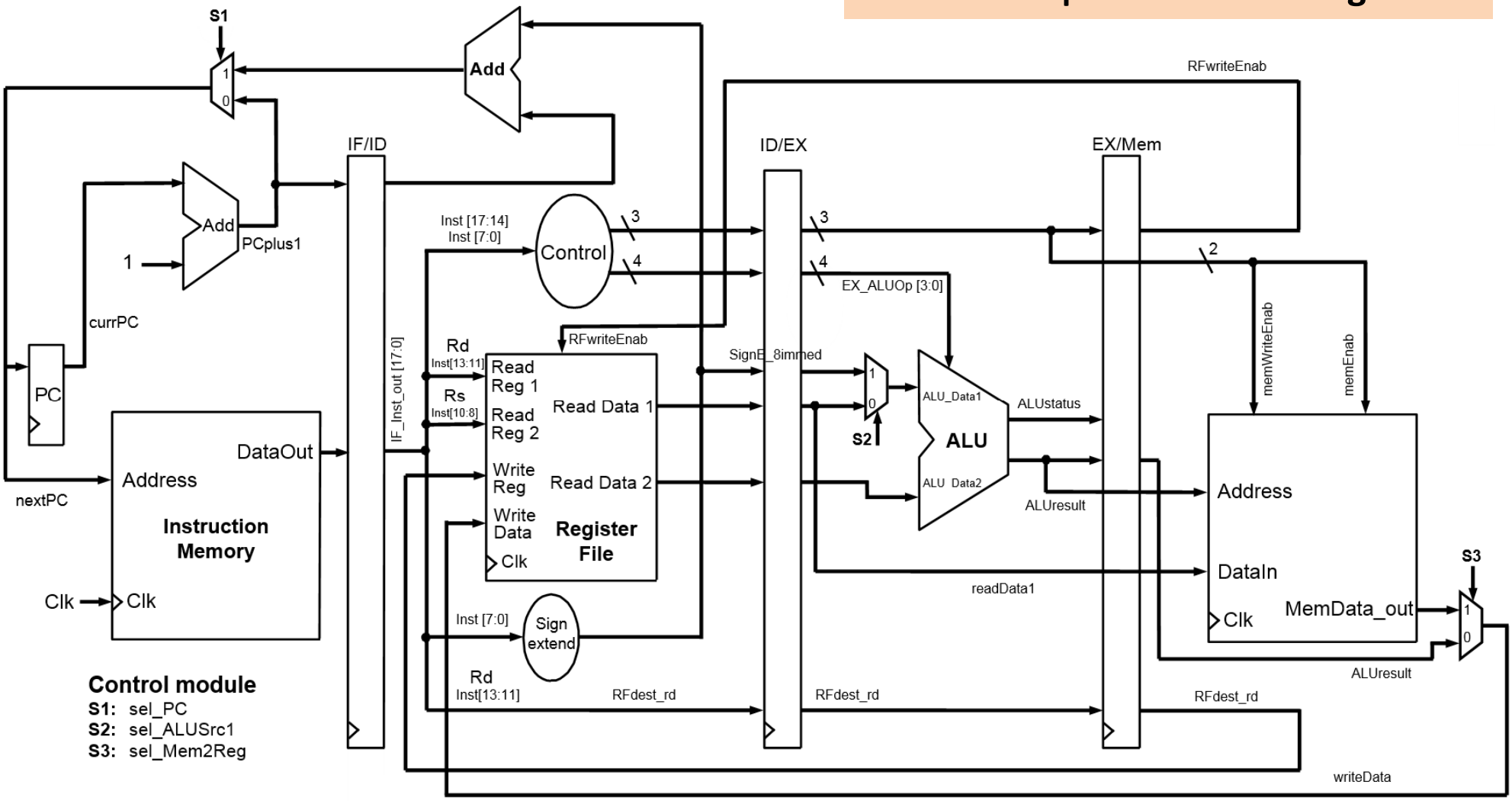






The uP16 System

Note: **Runs at 155 MHz on Spartan 6**
Critical path is in the ID stage

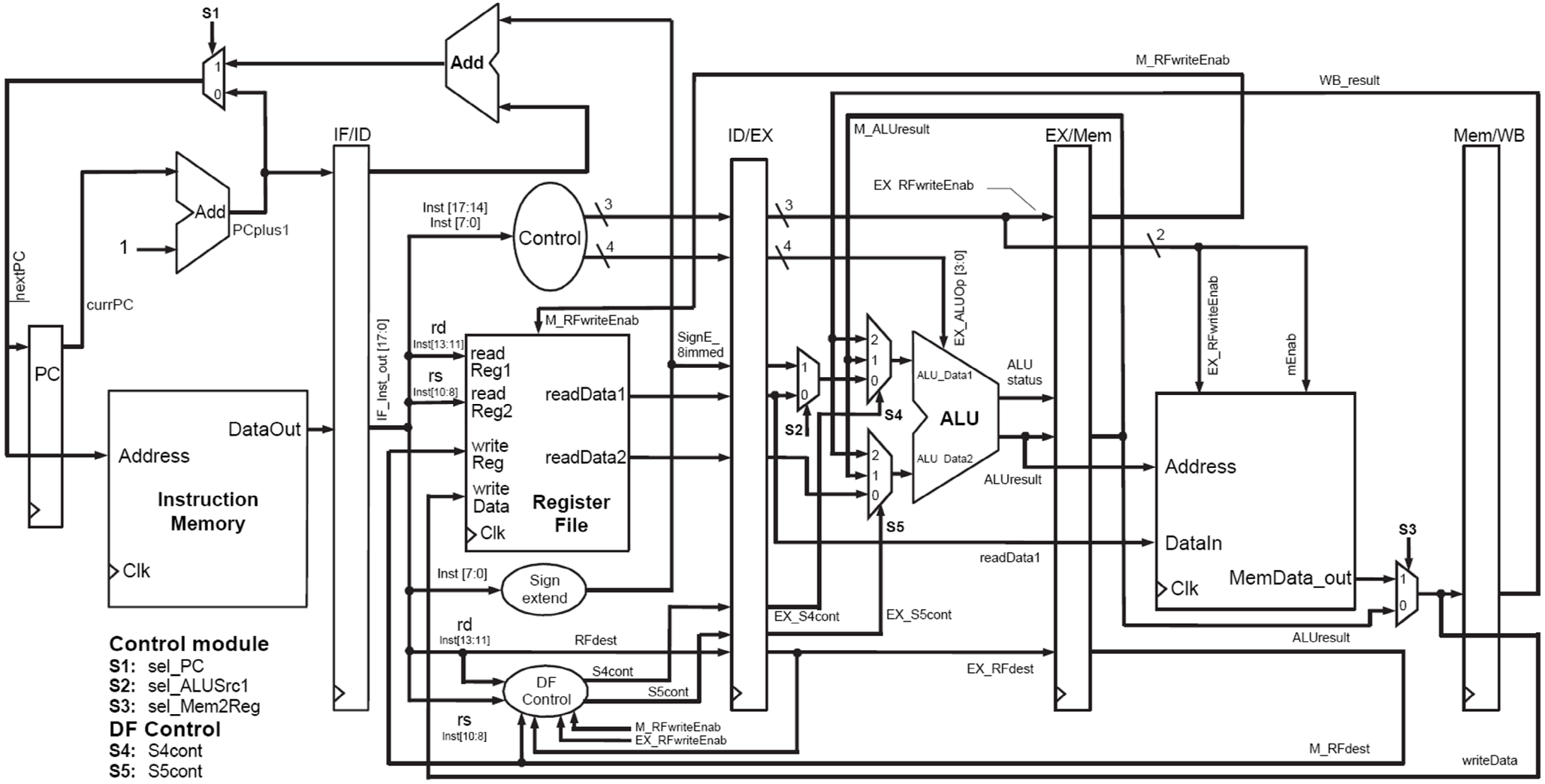


Pipeline hazards

- Pipeline Hazards
  - structural hazards: attempt to use the same resource two different ways at the same time
    - Only one memory system and we want to access data and instruction memory in same cycle.
  - control hazards: attempt to make a decision before a condition is evaluated
    - branch instructions
  - data hazards: attempt to use item before it is ready
    - An instruction depends on the result of a prior instruction still in the pipeline
- You can always resolve hazards by waiting
  - pipeline control must detect the hazard
  - take action (or delay action) to resolve the hazard
- Waiting slows the processor. So waiting is not a good idea.
  - But sometimes it is unavoidable

Final System (data forwarding)

Note: **Still runs at ~155 MHz**
Critical path is still in ID stage.



Discussion

1) why do we need a branch delay.

Consider

PC = 5	nop
= 6	beq $1, $1, #2
= 7	add ?
= 8	and ?
= 9	or ?
= 10	sub

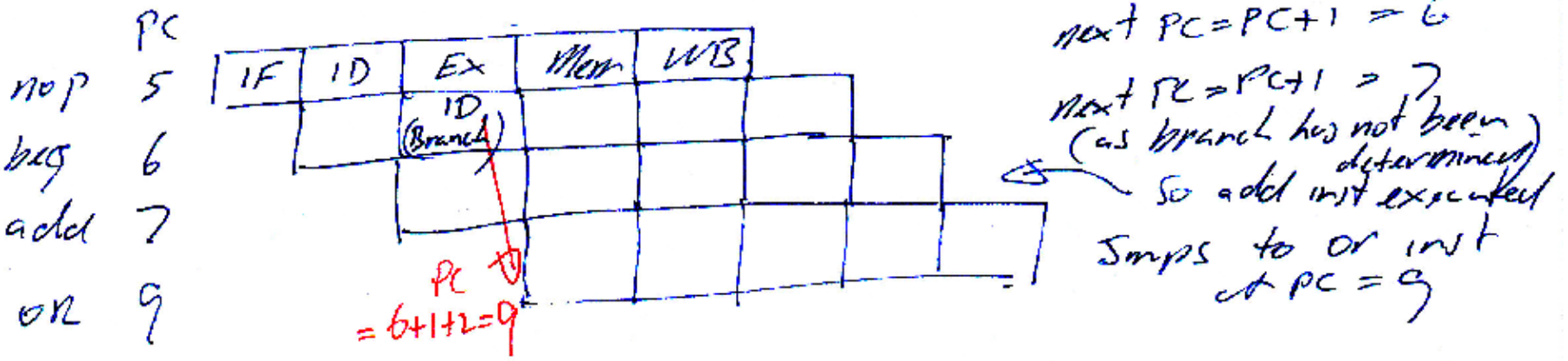
beq will always be Taken

new pc = Curr PC+1 + offset

New PC = 6 + 1 + 2 = 9

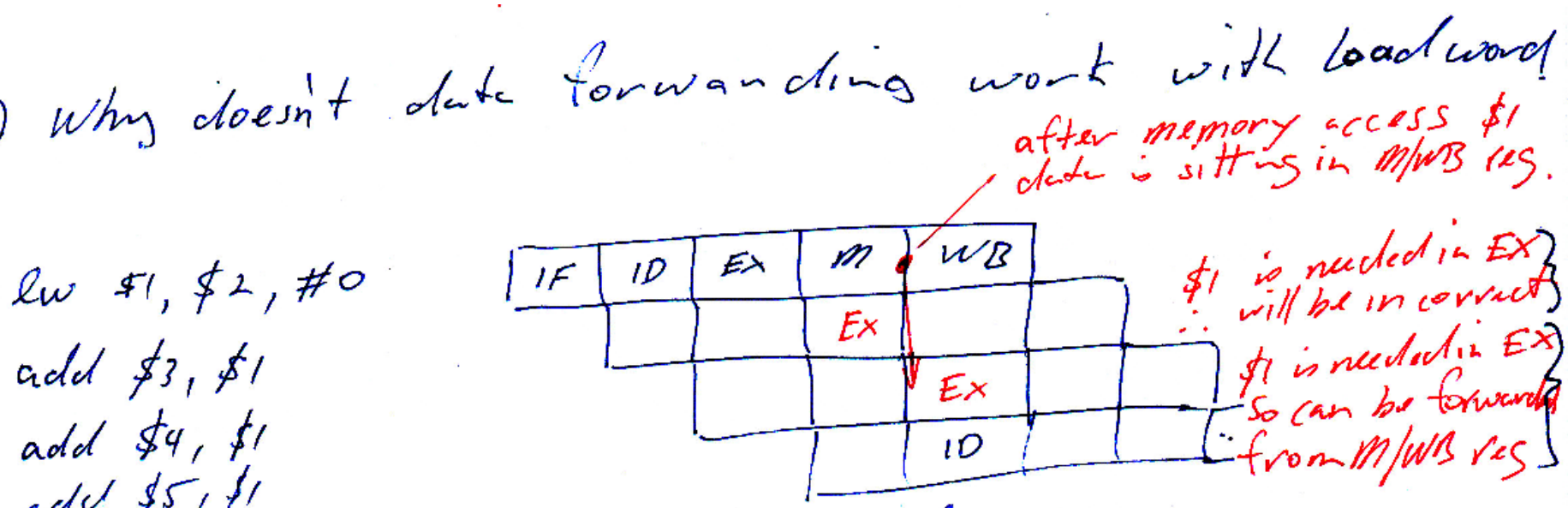
∴ 4th or instruction will be executed.

But what actually happens



Discussion

2) Why doesn't data forwarding work with load word?



∴ must have delay of 1 cycle before can use the result from a LW.

after 2 cycles, the "add $5, $1" instruction can access $1 from the RF. (no WB occurs at the start of the WB phase)

Hence, need a **load delay slot** after a Load Word.
This could be a NOP or a non-dependant instruction.

Exam: Solutions

AY18-19 Sem 2

Sketch the structural modifications that need to be made to the processor to implement the following jump instructions.

```
jmp     target;   // PC = zero_ext(target);
jlr     rd, rs;   // RF[rd] = (PC+1); PC = RF[rs];
```

13 marks

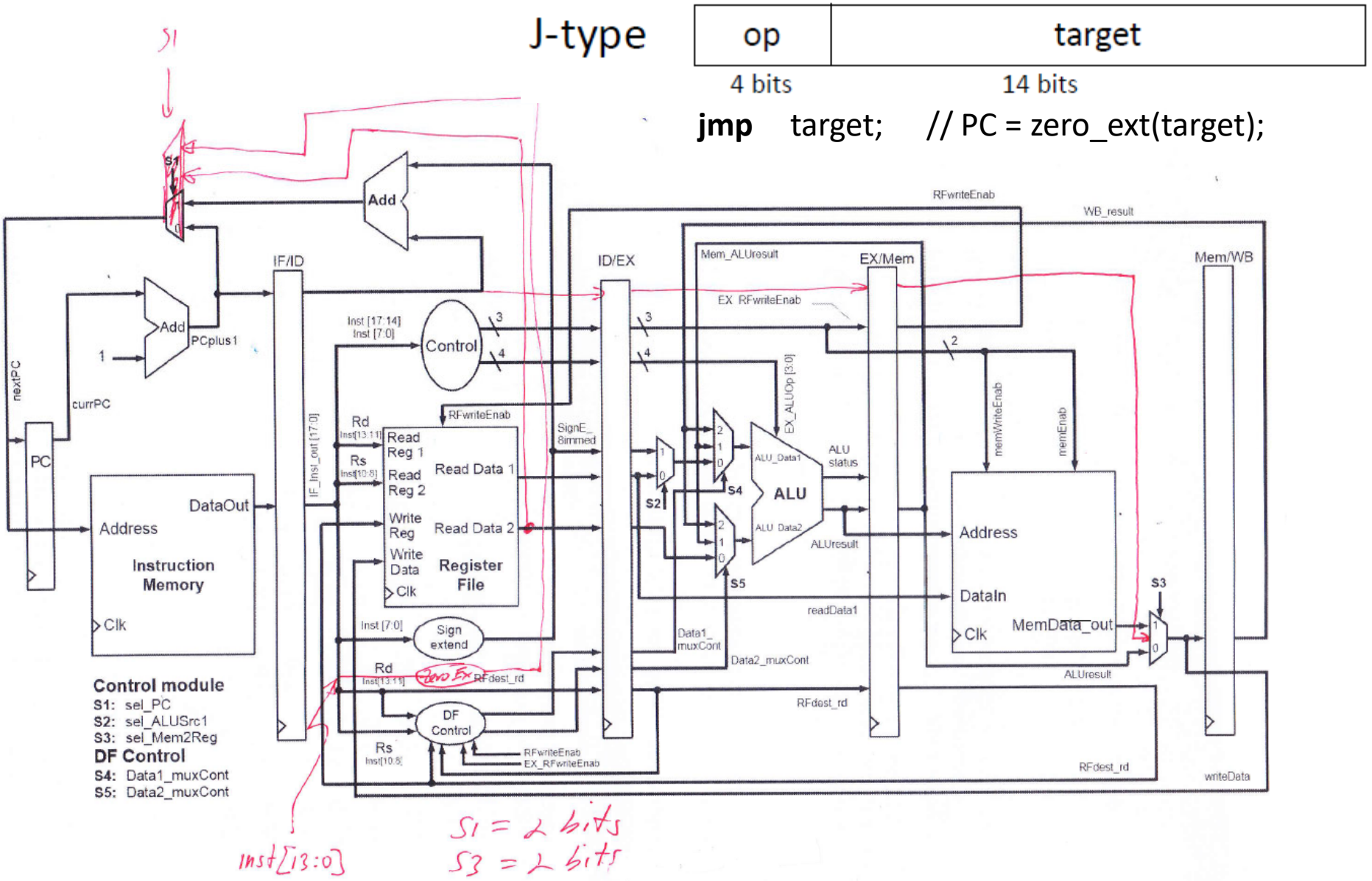
where **jmp** and **jlr** are the *jump* and *jump and link register* instructions.

Exam: Solutions

J-type

op	target
4 bits	14 bits

jmp target; // PC = zero_ext(target);

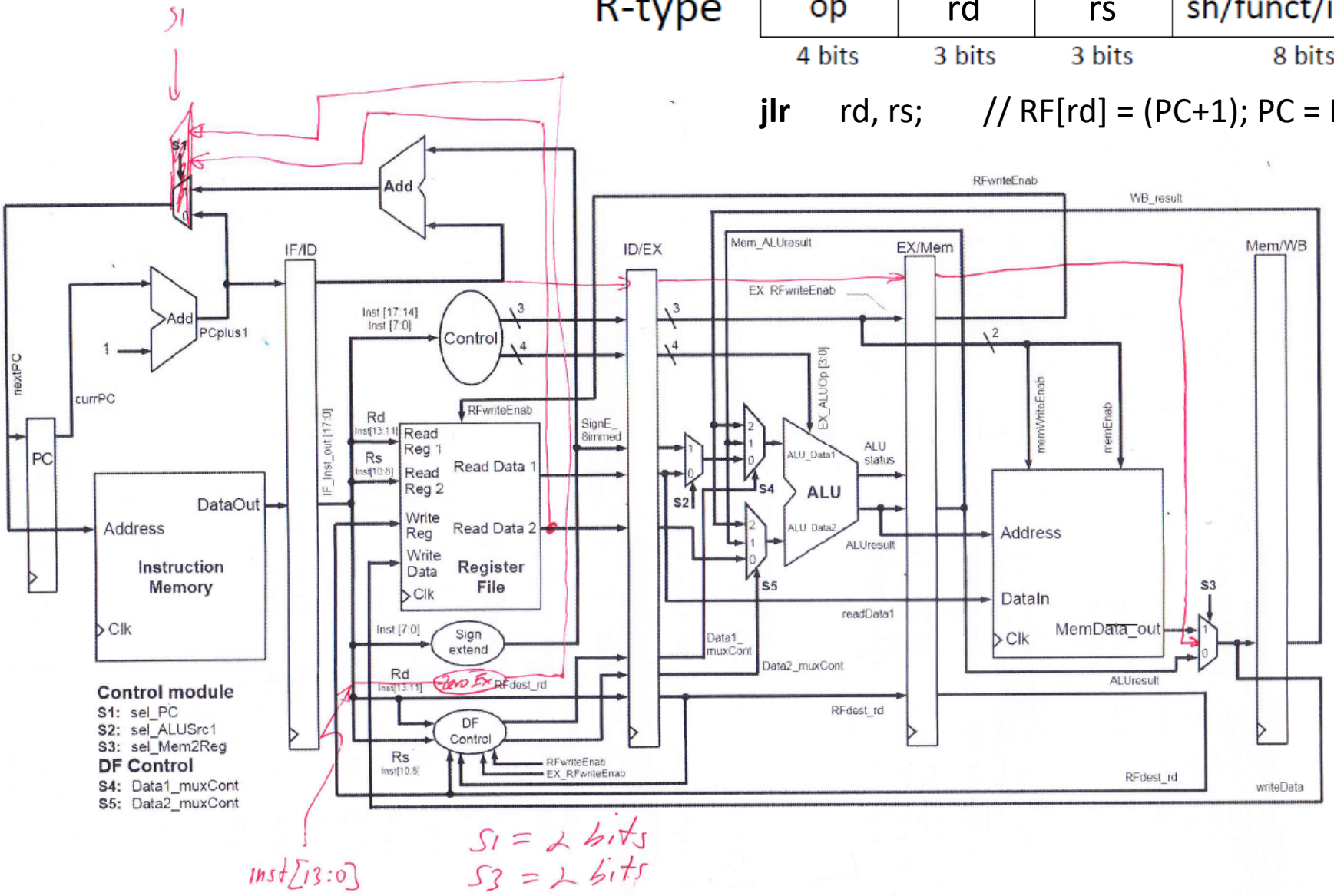


Exam: Solutions

R-type

op	rd	rs	sh/funct/immed
4 bits	3 bits	3 bits	8 bits

jlr rd, rs; // RF[rd] = (PC+1); PC = RF[rs];



AY1718 S2 (Apr 2018): Solutions

Given the block diagram and instruction set of a simple processor with data forwarding disabled (by setting the **S4** and **S5** control signals to "0")

1. What are $2 and $3 immediately after the execution of the last **nop** instruction in the sequence of instructions shown below.
2. How many times is the **bgt** instruction executed?

You may assume that the initial register file (RF) contents ($0 to $7) are all zero.

15 marks

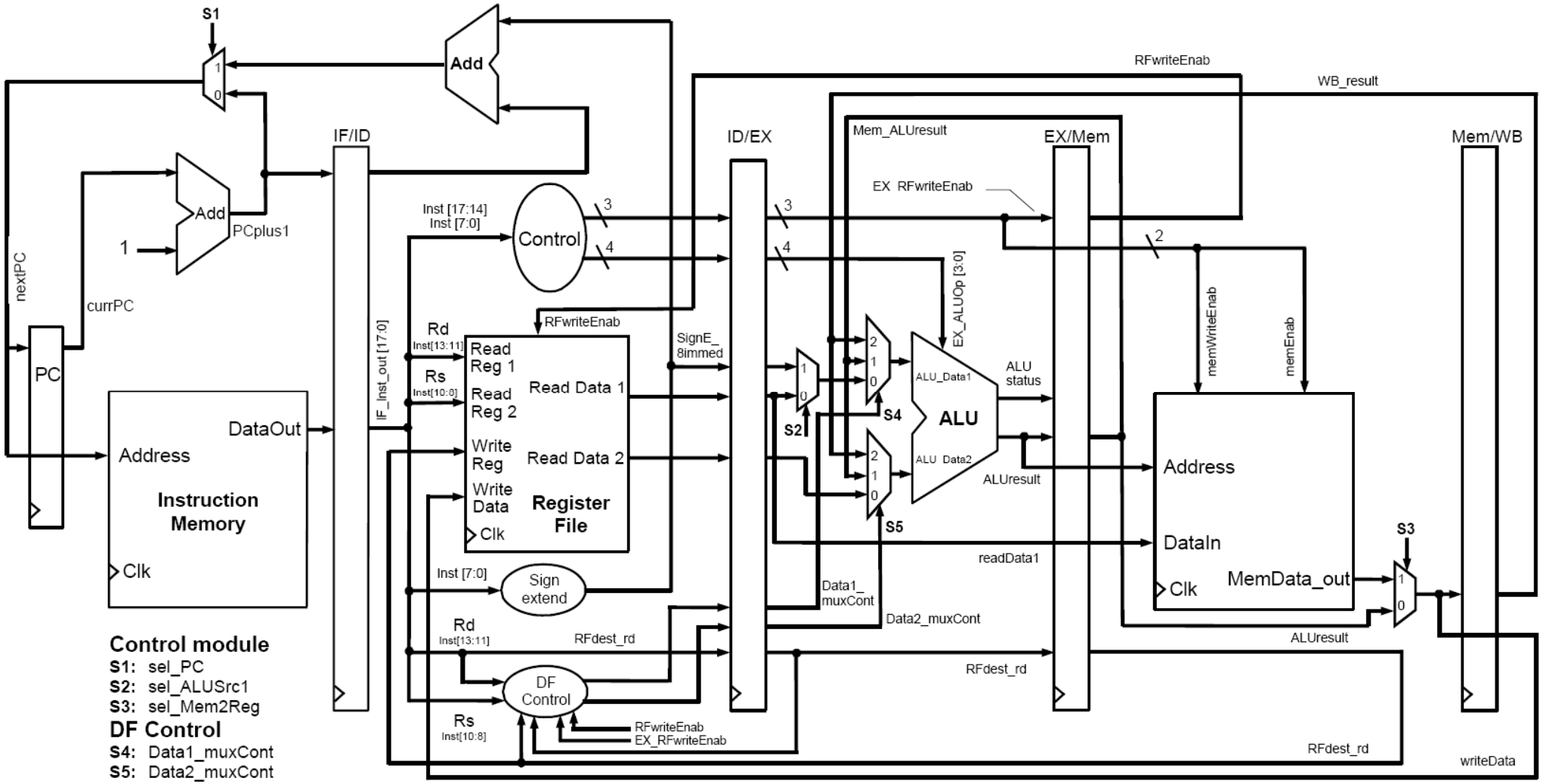
```
        lli   $1, #0
        lli   $3, #0
        lli   $2, #0
        nop
Loop:   bgt   $3, $1, #3      // the PC moves to "lli $1, #1" if branch taken
        addi  $2, $2, #1
        beg   $0, $0, #-3     // same as "Branch Always Loop"
        add   $3, $2
        lli   $1, #1
        nop
        nop
        nop
```

Remember, the instruction immediately following the **branch** instruction is always executed

Instruction Set

Name	op	Funct/ Immed	Instruction	Effect(s)
nop	0x00	0x00	**nop**	Do nothing (instruction word = 0)
add	0x00	0x01	**add** *rd, rs*	RF[rd] = RF[rd] + RF[rs];
sub	0x00	0x02	**sub** *rd, rs*	RF[rd] = RF[rd] - RF[rs];
and	0x00	0x03	**and** *rd, rs*	RF[rd] = RF[rd] **and** RF[rs];
or	0x00	0x04	**or** *rd, rs*	RF[rd] = RF[rd] **or** RF[rs];
nand	0x00	0x05	**nand** *rd, rs*	RF[rd] = RF[rd] **nand** RF[rs];
nor	0x00	0x06	**nor** *rd, rs*	RF[rd] = RF[rd] **nor** RF[rs];
xor	0x00	0x07	**xor** *rd, rs*	RF[rd] = RF[rd] **xor** RF[rs];
not	0x00	0x08	**not** *rd, rs*	RF[rd] = **not** RF[rs];
mov	0x00	0x0A	**mov** *rd, rs*	RF[rd] = RF[rs];
lw	0x01	--	**lw** *rd, rs, immed*	RF[rd] <- memory(RF[rs]+immed);
sw	0x02	--	**sw** *rd, rs, immed*	RF[rd] -> memory(RF[rs]+immed);
lli	0x03	immed	**lli** *rd, rs, immed*	RF[rd] = sign_ext(immed); (*rs* is unused)
lui	0x04	immed	**lui** *rd, rs, immed*	RF[rd] = {immed, RF[rs][7:0]}
addi	0x05	immed	**addi** *rd, rs, immed*	RF[rd] = RF[rs] + sign_ext(immed);
	0x6-8			**Reserved**
beq	0x08	immed	**beq** *rd, rs, immed*	if (RF[rd] = = RF[rs]) pc += sign_ext(immed) + 1;
bne	0x09	immed	**bne** *rd, rs, immed*	if (RF[rd] != RF[rs]) pc += sign_ext(immed) + 1;
blt	0x0A	immed	**bltz** *rd, rs, immed*	if (RF[rd] < RF[rs]) pc += sign_ext(immed) + 1;
bgt	0x0B	immed	**bgtz** *rd, rs, immed*	if (RF[rd] > RF[rs]) pc += sign_ext(immed) + 1;
	0xC-F			**Reserved**

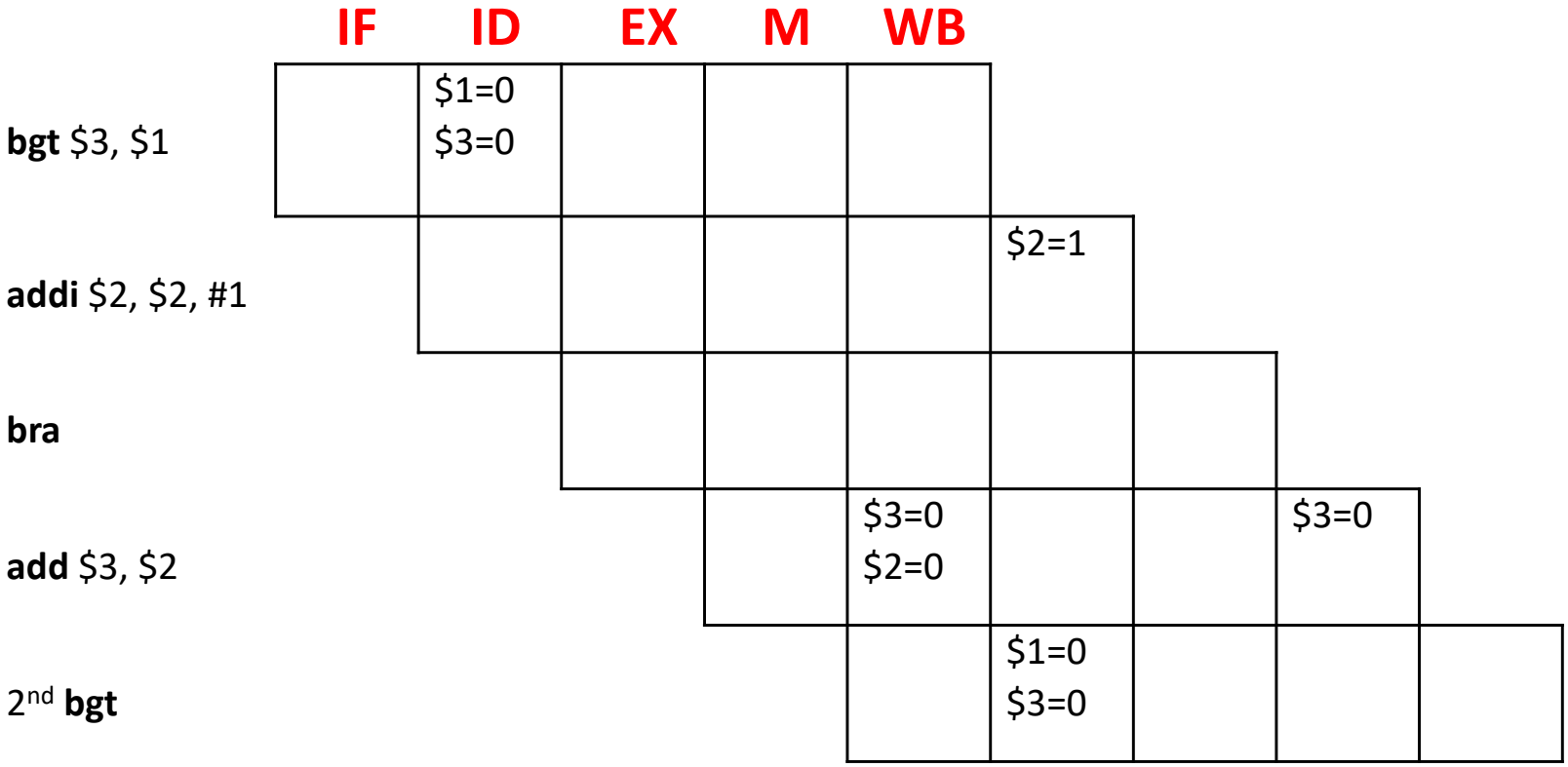
uP16 Block Diagram



Soln (No data forwarding)

- Firstly, by the time $1, $3 and $2 are used after the three **lli** instructions, the contents of each have been written to the RF
- As $3 = $1 = 0, the **bgt** instruction is not taken. Sequence is:

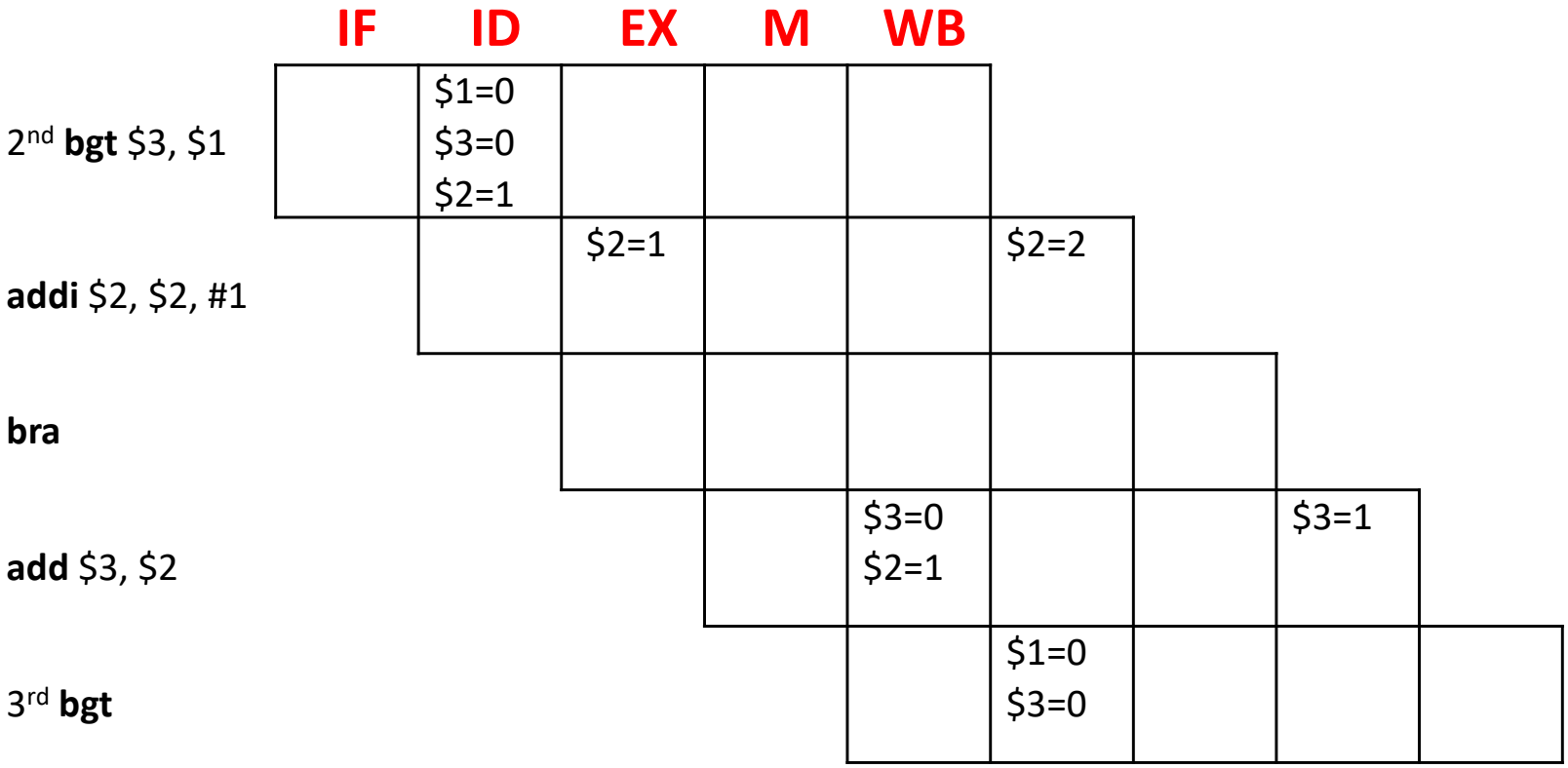
First iteration:



Soln (No data forwarding)

- Firstly, by the time $1, $3 and $2 are used after the three **lli** instructions, the contents of each have been written to the RF
- As $3 = $1 = 0, the **bgt** instruction is not taken. Sequence is:

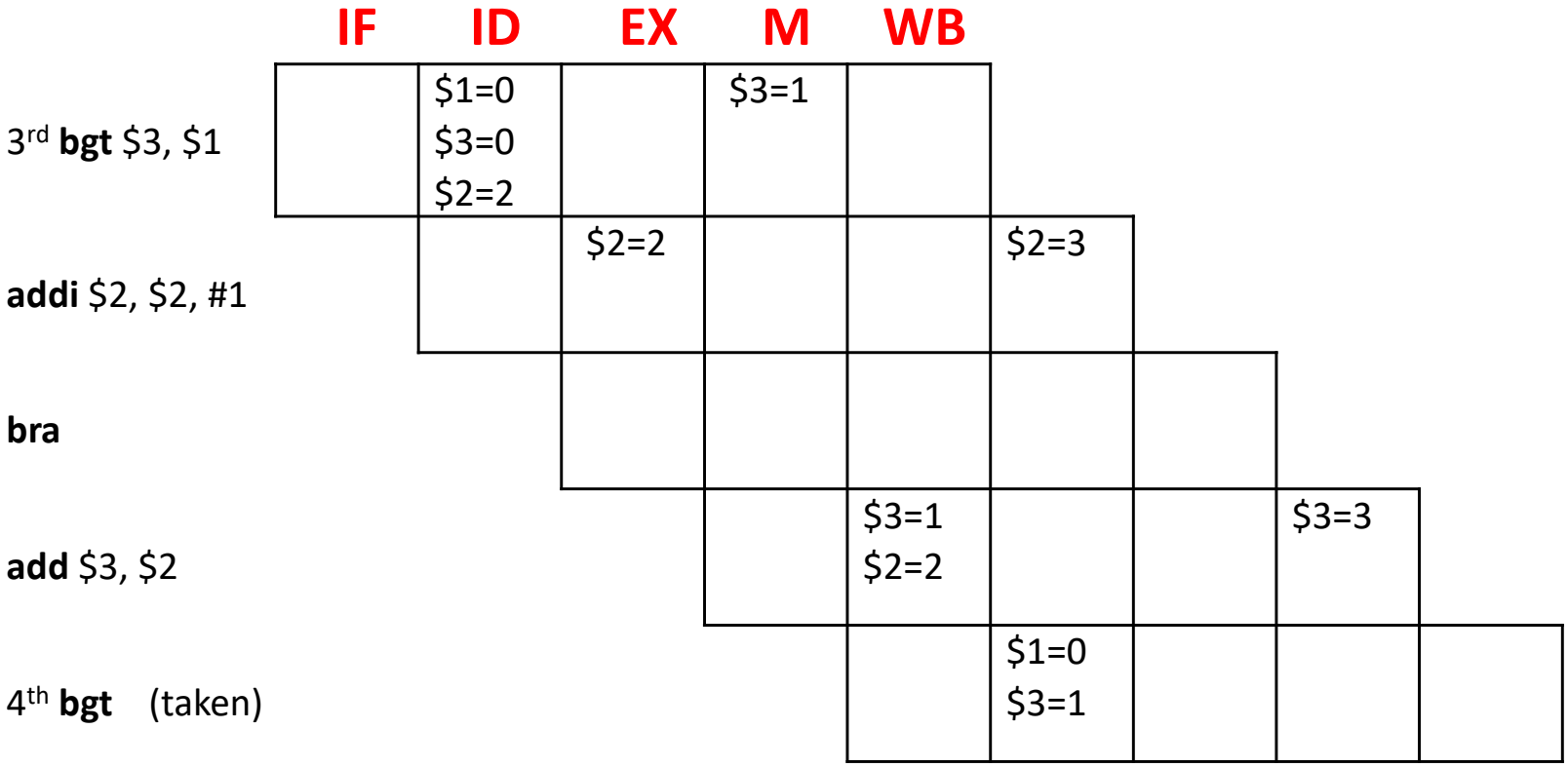
Second iteration:



Soln (No data forwarding)

- Firstly, by the time $1, $3 and $2 are used after the three **lli** instructions, the contents of each have been written to the RF
- As $3 = $1 = 0, the **bgt** instruction is not taken. Sequence is:

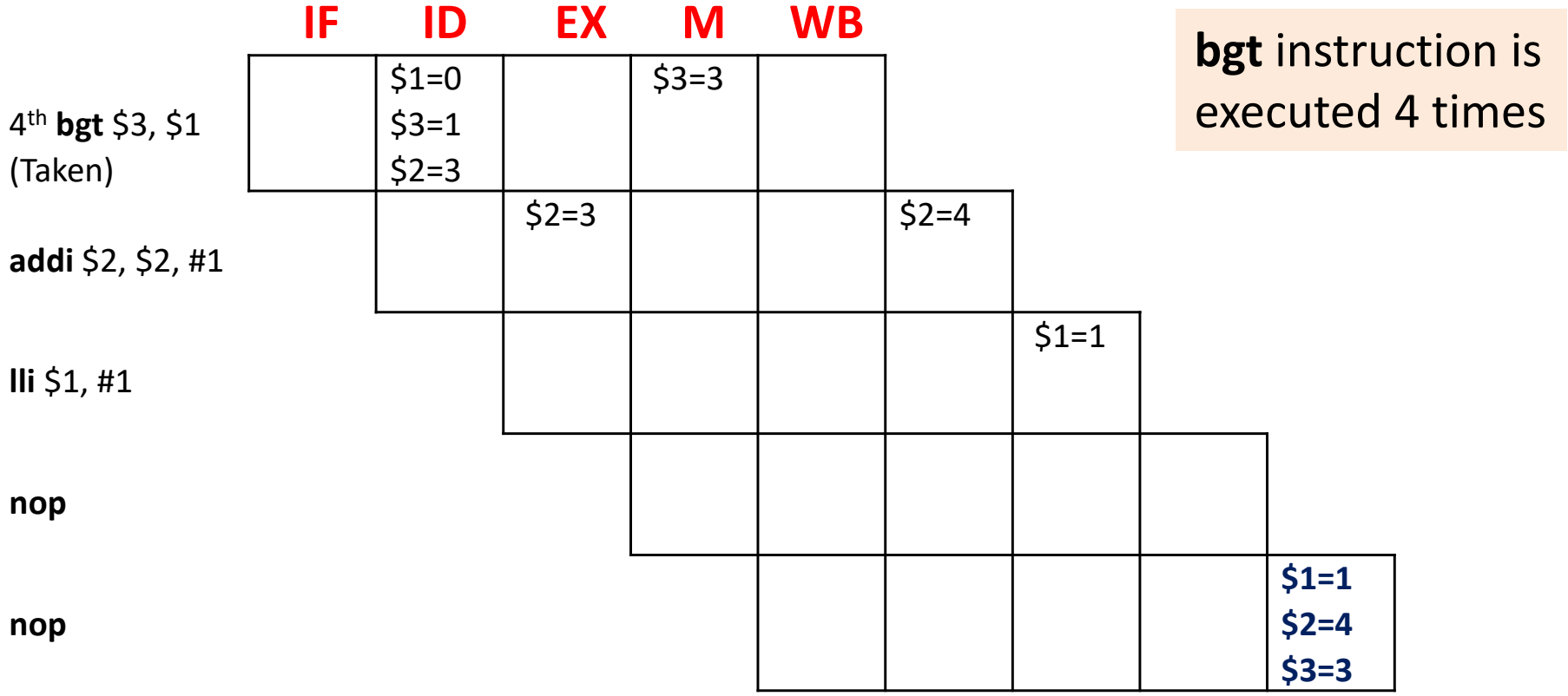
Third iteration:



Soln (No data forwarding)

- Firstly, by the time $1, $3 and $2 are used after the three **lli** instructions, the contents of each have been written to the RF
- As $3 = 1, $1 = 0, the **bgt** instruction is taken. Sequence is:

Final:



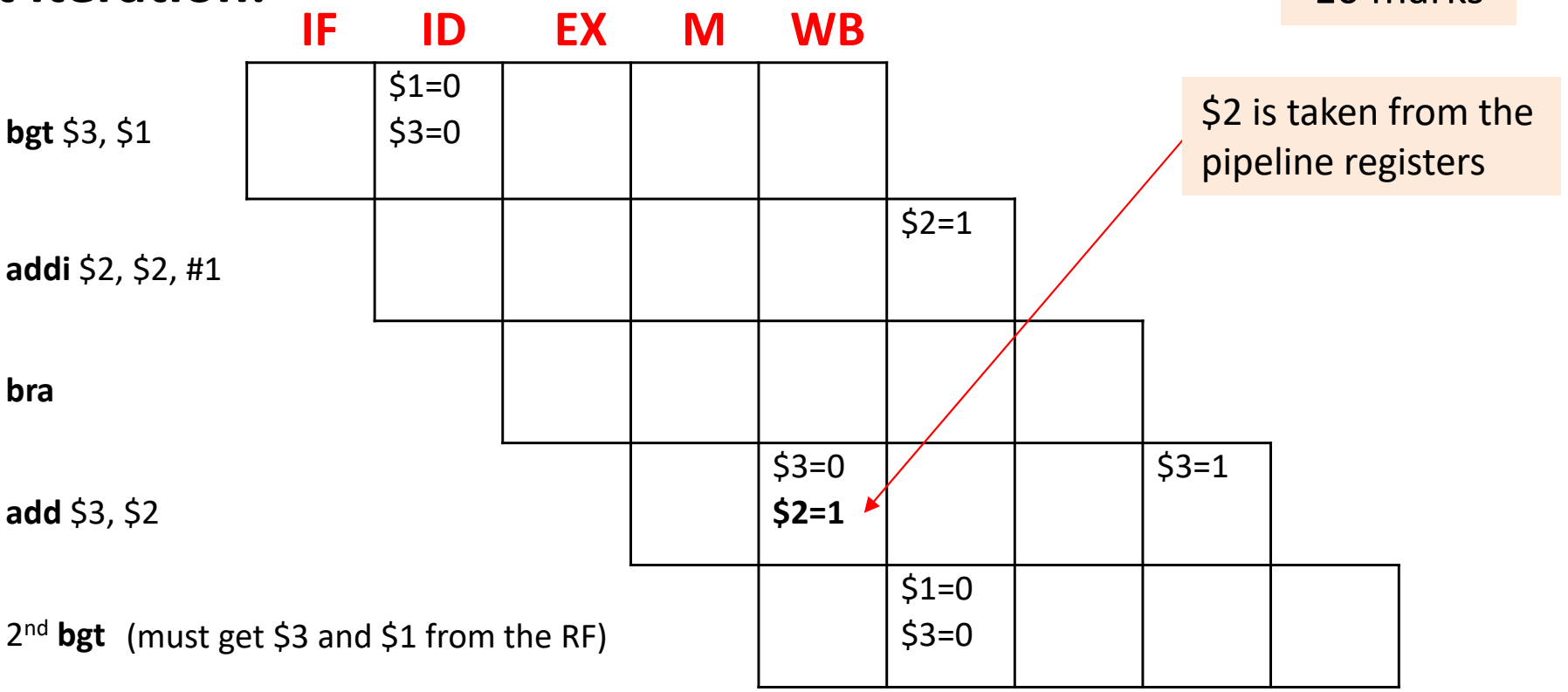
bgt instruction is executed 4 times

Soln (Data forwarding)

- Now with data forwarding enabled. Data forwarding will impact the calculation of $3=$3+$2, but not the use in **bgt**.
- Again, as $3 = $1 = 0, the first **bgt** instruction is not taken.

First iteration:

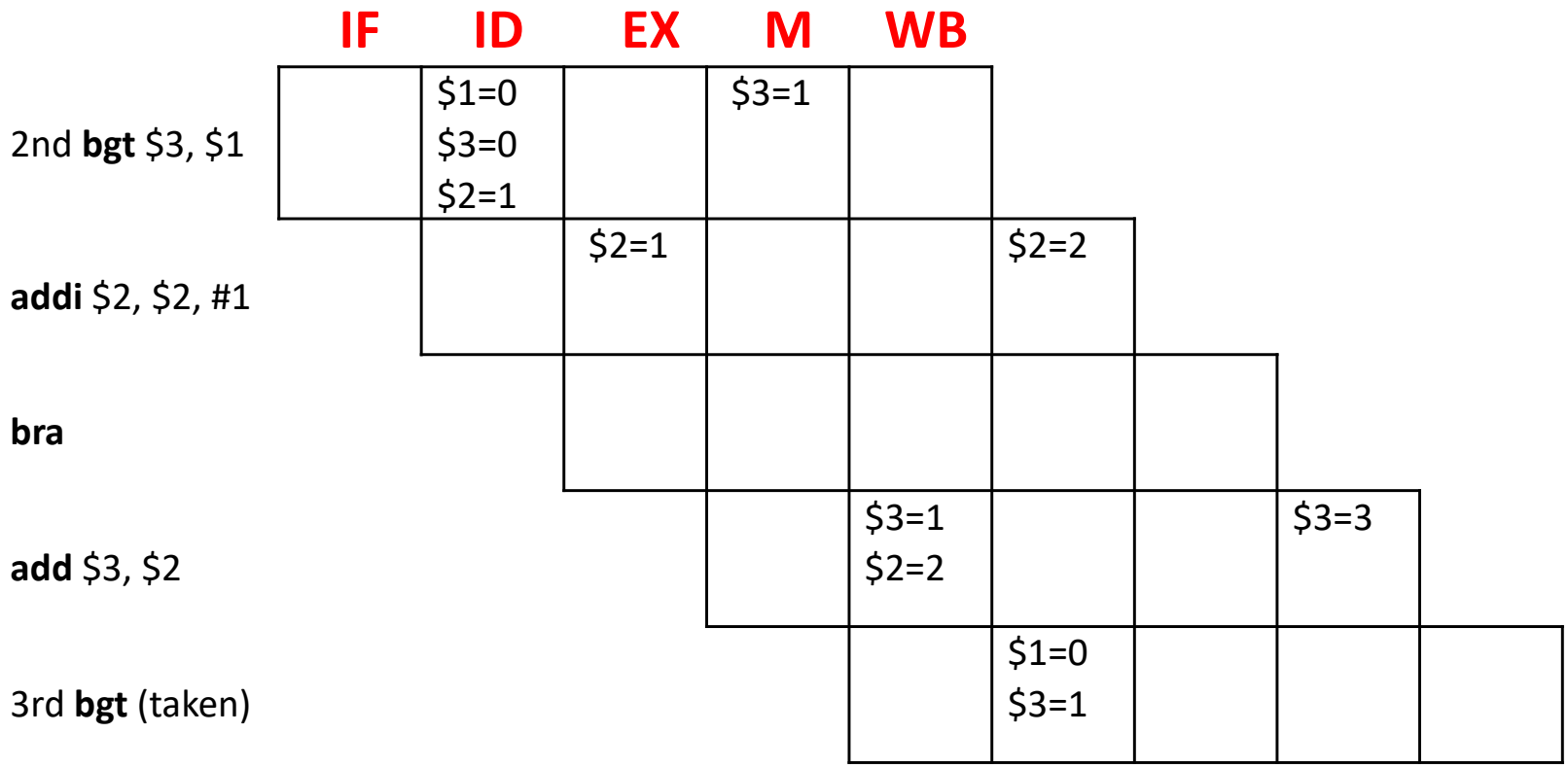
10 marks



Soln (Data forwarding)

- Now with data forwarding enabled. Data forwarding will impact the calculation of $3=$3+$2, but not the use in **bgt**.

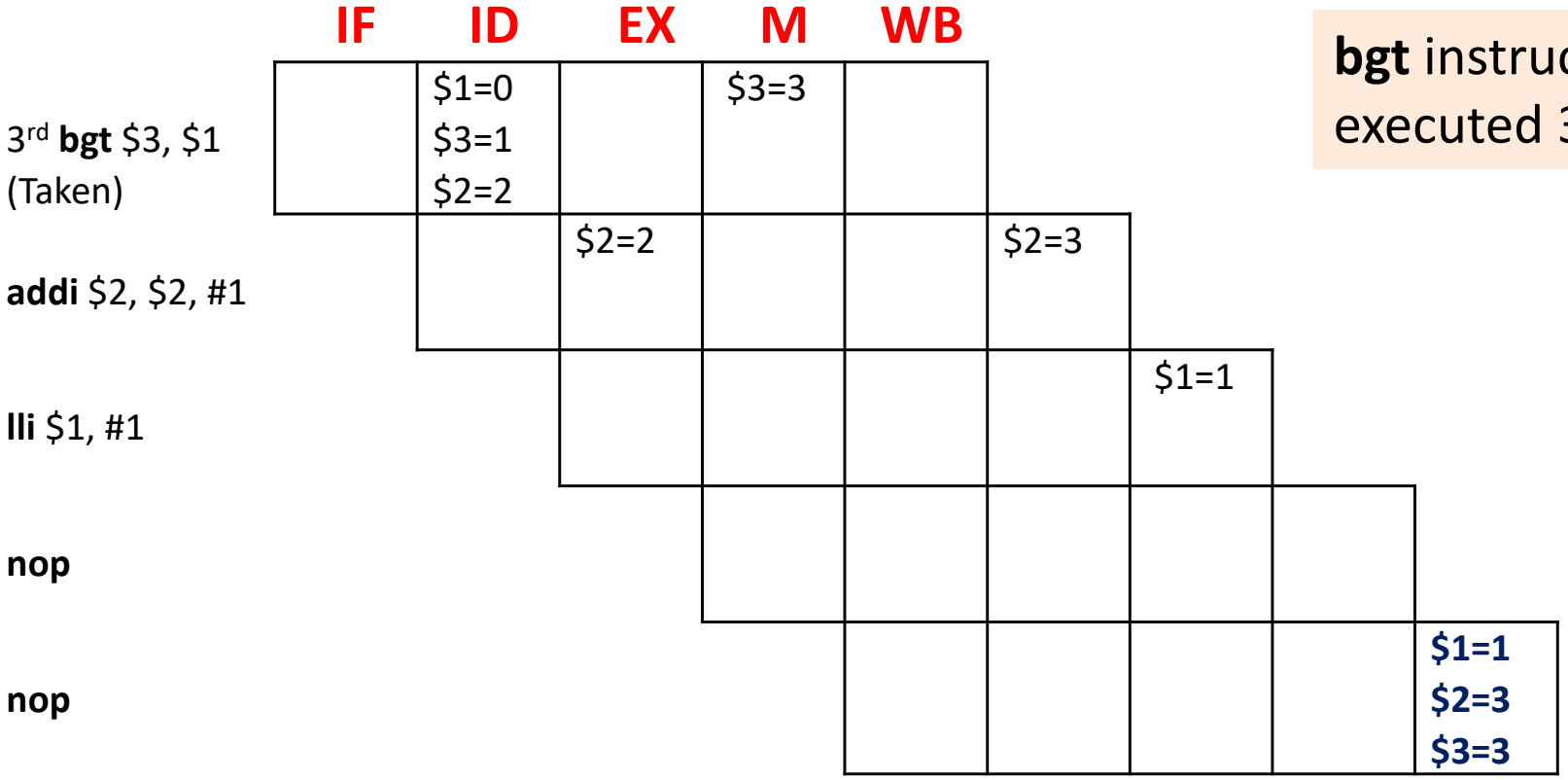
Second iteration:



Soln (Data forwarding)

- Now with data forwarding enabled. Data forwarding will impact the calculation of $3=$3+$2, but not the use in **bgt**.
- As $3 = 1, $1 = 0, the **bgt** instruction is taken.

Third iteration:



bgt instruction is executed 3 times

End of EC4 (Mod 8)